

Jahangirnagar University

Department of Computer Science and Engineering 2nd Year 1st Semester B.Sc. (Hons.) Final Examination – 2024

Course Title: Algorithms-I

Course Code: CSE-209

Time: 3 Hours

Full Marks: 60

Complete Solutions

Contents

Section A (CO1)	2
Section B (CO2)	4
Section C (CO3)	6
Section D (CO4)	10
Section E (CO5)	14

SECTION A (CO1) – Answer All Questions

Q1(a) – Define Algorithm; State Key Characteristics

Algorithm: An algorithm is a finite, well-defined sequence of unambiguous instructions that solves a computational problem, taking one or more inputs and producing one or more outputs, and always terminating in a finite number of steps.

Key Characteristics:

1. **Input:** Zero or more well-defined inputs.
2. **Output:** At least one output that relates correctly to the inputs.
3. **Definiteness:** Each step must be clear and unambiguous.
4. **Finiteness:** Must terminate after a finite number of steps.
5. **Effectiveness:** Each step must be basic enough to be performed (in principle) by pen and paper.

Q1(b) – Any Decision Tree Sorting n Elements Has $\Omega(n \lg n)$ Comparisons

Proof:

A comparison-based sorting algorithm can be represented as a *decision tree* where:

- Each internal node represents a comparison between two elements.
- Each leaf represents a possible permutation (outcome) of the n elements.

Step 1 – Number of leaves. There are exactly $n!$ possible orderings of n elements, so the decision tree must have at least $n!$ leaves.

Step 2 – Height of the tree. A binary tree of height h has at most 2^h leaves. Therefore:

$$2^h \geq n! \implies h \geq \log_2(n!).$$

Step 3 – Estimate $\log_2(n!)$. By Stirling's approximation, $n! \geq (n/e)^n$, so:

$$\log_2(n!) \geq n \log_2 n - n \log_2 e = \Omega(n \log n).$$

More directly:

$$\log_2(n!) = \sum_{k=1}^n \log_2 k \geq \sum_{k=\lceil n/2 \rceil}^n \log_2 k \geq \frac{n}{2} \cdot \log_2 \left(\frac{n}{2} \right) = \frac{n}{2} (\log_2 n - 1) = \Omega(n \log n).$$

Conclusion. The height $h \geq \Omega(n \log n)$, and since height equals the worst-case number of comparisons, any comparison-based sorting algorithm requires $\Omega(n \lg n)$ comparisons in the worst case. $\square$

Q1(c) – Properties of Greedy Algorithm

1. **Greedy-Choice Property:** A globally optimal solution can be reached by making the locally optimal (greedy) choice at each step.
2. **Optimal Substructure:** The optimal solution to the problem contains optimal solutions to its sub-problems.
3. **Feasibility:** Each chosen element must satisfy the constraints of the problem.
4. **Irrevocability:** Once a choice is made, it is never reconsidered or reversed.
5. **No Backtracking:** The algorithm proceeds forward without revisiting past decisions.

Q1(d) – Define with Example: Stable Sort & In-Place Sort

i. Stable Sort: A sorting algorithm is *stable* if it preserves the *relative order* of records with equal keys.

Example: Consider the array $[(3, A), (1, B), (3, C), (2, D)]$ sorted by the first key. A stable sort produces $[(1, B), (2, D), (3, A), (3, C)]$ – $(3, A)$ remains before $(3, C)$. **Merge Sort** and **Insertion Sort** are stable.

ii. In-Place Sort: A sorting algorithm is *in-place* if it uses $O(1)$ auxiliary space beyond the input array (it rearranges elements within the original array).

Example: **Insertion Sort** – at each step, element $A[i]$ is shifted leftward into its correct position using only a temporary variable. No extra array is needed. **Heap Sort** and **Selection Sort** are also in-place.

Q1(e) – Greedy Coin Change Algorithm

Coin denominations: $\{1, 2, 5, 10, 20\}$. Goal: give change for amount N using the minimum number of coins.

Algorithm (Greedy):

1. Sort coins in descending order: $[20, 10, 5, 2, 1]$.
2. Initialize: $count \leftarrow 0, rem \leftarrow N$.
3. For each coin c (largest to smallest):
 - $count += \lfloor rem/c \rfloor$
 - $rem \leftarrow rem \bmod c$
4. Return $count$.

Example: $N = 36$: take $1 \times 20, 1 \times 10, 1 \times 5, 0 \times 2, 1 \times 1 \Rightarrow 4$ coins.

Note: Greedy works correctly for this canonical coin system $\{1, 2, 5, 10, 20\}$ because it satisfies the greedy-choice property (each denomination is a multiple or subset-sum of smaller ones). The complexity is $O(k)$ where k is the number of denominations.

Q1(f) – Quick Sort vs. Randomized Quick Sort

Property	Quick Sort	Randomized Quick Sort
Pivot selection	Fixed (e.g., last element)	Random element
Worst-case time	$O(n^2)$ – triggered by sorted/reverse	$O(n^2)$ but probability $\rightarrow 0$
Average-case time	$O(n \log n)$	$O(n \log n)$ (expected, for all inputs)
Best-case time	$O(n \log n)$	$O(n \log n)$
Susceptibility	Adversarial inputs (sorted array)	No adversarial input – random pivot
Space	$O(\log n)$ avg (stack)	$O(\log n)$ avg (stack)
Stable?	No	No
In-place?	Yes	Yes

Limitation of Quick Sort: On an already-sorted input, the fixed pivot (last element) always produces the worst-case $O(n^2)$ partition.

Advantage of Randomized: By choosing the pivot uniformly at random, the expected runtime is $O(n \log n)$ regardless of input order, with negligible probability of worst-case behavior.

SECTION B (CO2) – Answer Any Three

Q2(a) – Merge $a[m]$ (Ascending) and $b[n]$ (Descending) into $C[n+m]$ Ascending

Idea: a is ascending so traverse left to right. b is descending, so the smallest element of b is at index n (the last position). Traverse b from right to left.

Pseudocode:

```

MERGE-ASC-DESC(a[1..m], b[1..n], C[1..m+n]):
    i = 1          // pointer for a (start)
    j = n          // pointer for b (start from end = smallest)
    k = 1          // pointer for output C
    while i <= m and j >= 1:
        if a[i] <= b[j]:
            C[k] = a[i]; i = i + 1
        else:
            C[k] = b[j]; j = j - 1
        k = k + 1
    while i <= m:
        C[k] = a[i]; i = i + 1; k = k + 1
    while j >= 1:
        C[k] = b[j]; j = j - 1; k = k + 1

```

Example: $a = [1, 3, 5]$ (ascending), $b = [6, 4, 2]$ (descending).

Merge with j starting at 3 (value 2): compare $a[1] = 1$ vs $b[3] = 2 \Rightarrow$ take 1.

Compare $a[2] = 3$ vs $b[3] = 2 \Rightarrow$ take 2. Compare $a[2] = 3$ vs $b[2] = 4 \Rightarrow$ take 3. Etc.

Result: $C = [1, 2, 3, 4, 5, 6]$ ✓

Time complexity: $O(m + n)$. **Space:** $O(m + n)$ for output array C .

Q2(b) – Master Method for Recurrences

Recall the Master Theorem: for $T(n) = aT(n/b) + f(n)$ with $a \geq 1, b > 1$:

- **Case 1:** $f(n) = O(n^{\log_b a - \varepsilon}) \Rightarrow T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** $f(n) = \Theta(n^{\log_b a} \log^k n) \Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
- **Case 3:** $f(n) = \Omega(n^{\log_b a + \varepsilon})$ and regularity holds $\Rightarrow T(n) = \Theta(f(n))$.

i. $T(N) = 2T\left(\frac{N}{2}\right) + \sqrt{N}$

Here $a = 2, b = 2, f(n) = n^{1/2}$.

$$n^{\log_b a} = n^{\log_2 2} = n^1.$$

Since $f(n) = n^{1/2} = O(n^{1-\varepsilon})$ for $\varepsilon = \frac{1}{2} > 0$, **Case 1** applies.

$$T(N) = \Theta(N)$$

ii. $T(N) = 4T\left(\frac{N}{2}\right) + N^2 \lg N$

Here $a = 4, b = 2, f(n) = n^2 \lg n$.

$$n^{\log_b a} = n^{\log_2 4} = n^2.$$

Since $f(n) = n^2 \lg n = \Theta(n^{\log_b a} \cdot \log^1 n)$, **Case 2** applies with $k = 1$.

Regularity check: $a f(n/b) = 4 \cdot (n/2)^2 \lg(n/2) = n^2 (\lg n - 1) \leq c n^2 \lg n$ for c close to 1 and large n . ✓

$$T(N) = \Theta(N^2 \log^2 N)$$

Q2(c) – BFS from Vertex C

Adjacency lists (given): A: B,C,D,H; B: A,C,E,H; C: A,B,G; D: A,F; E: B,G; F: D; G: C,E; H: A,B.

Process neighbors from smallest to largest vertex number. Mark as visited when enqueued.

BFS execution:

- **Dequeue C** (dist 0): neighbors A,B,G $\rightarrow$ enqueue A,B,G.
- **Dequeue A** (dist 1): neighbors B(visited),C(visited),D,H $\rightarrow$ enqueue D,H.
- **Dequeue B** (dist 1): neighbors A(v),C(v),E,H(v) $\rightarrow$ enqueue E.
- **Dequeue G** (dist 1): neighbors C(v),E(v) $\rightarrow$ none.
- **Dequeue D** (dist 2): neighbors A(v),F $\rightarrow$ enqueue F.
- **Dequeue H** (dist 2): neighbors A(v),B(v) $\rightarrow$ none.
- **Dequeue E** (dist 2): neighbors B(v),G(v) $\rightarrow$ none.
- **Dequeue F** (dist 3): neighbors D(v) $\rightarrow$ none.

Order	Vertex v	Edge (u, v) added	Distance	Parent
1 st	C	– (source)	0	–
2 nd	A	(C,A)	1	C
3 rd	B	(C,B)	1	C
4 th	G	(C,G)	1	C
5 th	D	(A,D)	2	A
6 th	H	(A,H)	2	A
7 th	E	(B,E)	2	B
8 th	F	(D,F)	3	D

Q2(d) – Graph-1: Topological Ordering and Properties

Reading Graph-1 from the figure: nodes A, B, C, D, E with directed edges. From the figure the visible edges and directions suggest: $C \rightarrow A$, $C \rightarrow D$, $A \rightarrow B$, $B \rightarrow E$, $D \rightarrow E$ (and edge weights 2,5,3,7,5 approx).

i. Topological Orderings:

For a DAG with edges $C \rightarrow A$, $C \rightarrow D$, $A \rightarrow B$, $B \rightarrow E$, $D \rightarrow E$:

In-degrees: $C = 0$, $A = 1$, $D = 1$, $B = 1$, $E = 2$.

Kahn's algorithm (process in-degree 0 nodes):

1. Start: $\{C\}$. Remove $C \rightarrow$ reduce in-degree of A and D . Both become 0.
2. Two choices: A or D next.

- **Order 1:** $C \rightarrow A \rightarrow D \rightarrow B \rightarrow E$
- **Order 2:** $C \rightarrow D \rightarrow A \rightarrow B \rightarrow E$

Two valid topological orderings: $\{C, A, D, B, E\}$ and $\{C, D, A, B, E\}$.

ii. Which properties apply to the graph?

Property	Answer	Reason
Complete	No	Not all pairs have edges in both directions
Directed	Yes	Edges have directions (arrows)
Strongly connected	No	No path from E back to C
Weighted	Yes	Edge weights are shown
Acyclic	Yes	No cycles $\Rightarrow$ it is a DAG
Weakly connected	Yes	All nodes connected ignoring direction

SECTION C (CO3) – Answer Any Three

Q3(a) – Max-Heap Construction

Given array (1-indexed):

Index	1	2	3	4	5	6	7	8	9	10	11	12
Data	80	60	40	8	65	22	28	74	6	12	2	13

Is this a Max-Heap at index 1?

Root is at index 1 with value **80**. Check the heap property downward:

- Index 2 (value 60): left child at index 4 (value 8) ✓; right child at index 5 (value **65**).
- $60 < 65$ **VIOLATION!**

No, this array does NOT represent a Max-Heap because the node at index 2 (value 60) has a child at index 5 (value 65) that is greater than it.

Building the Max-Heap using Max-Heapify:

We call MAX-HEAPIFY on all internal nodes from index $\lfloor n/2 \rfloor = 6$ down to 1.

Call 1 – Heapify(6): Value 22, children: index 12 (value 13). $22 > 13$ ✓ No swap.

Call 2 – Heapify(5): Value 65, children: index 10 (value 12), index 11 (value 2). $65 > 12, 65 > 2$ ✓ No swap.

Call 3 – Heapify(4): Value 8, children: index 8 (value **74**), index 9 (value 6). $8 < 74 \Rightarrow$ swap index 4 and 8. Array: $[\dots, \mathbf{74}, \dots, \mathbf{8}, \dots]$. Heapify(8): value 8, children at 16,17 (don't exist). Done.

Array after Call 3: $[80, 60, 40, \mathbf{74}, 65, 22, 28, \mathbf{8}, 6, 12, 2, 13]$

Call 4 – Heapify(3): Value 40, children: index 6 (22), index 7 (28). $40 > 22, 40 > 28$ ✓ No swap.

Call 5 – Heapify(2): Value 60, children: index 4 (**74**), index 5 (65). $60 < 74 \Rightarrow$ swap index 2 and 4. Array: $[80, \mathbf{74}, 40, \mathbf{60}, 65, 22, 28, 8, 6, 12, 2, 13]$. Heapify(4): value 60, children: index 8 (8), index 9 (6). $60 > 8, 60 > 6$ ✓ No swap.

Call 6 – Heapify(1): Value 80, children: index 2 (74), index 3 (40). $80 > 74, 80 > 40$ ✓ No swap.

Final Max-Heap array:

Index	1	2	3	4	5	6	7	8	9	10	11	12
Data	80	74	40	60	65	22	28	8	6	12	2	13

Total Max-Heapify calls = $\lfloor 12/2 \rfloor = 6$ (on indices 6, 5, 4, 3, 2, 1).

Q3(b) – Rock Climbing Problem (Maximum Path Sum in Grid)

Grid (row 0 = top):

$$\begin{pmatrix} 2 & 4 & 5 & 1 & 7 \\ 4 & 5 & 3 & 8 & 8 \\ 1 & 3 & 6 & 3 & 9 \\ 15 & 8 & 6 & 12 & 7 \end{pmatrix}$$

DP formulation: $dp[i][j] = \text{grid}[i][j] + \max(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])$
(boundary: only valid adjacent columns)

Row 0 (base):

2 4 5 1 7

Row 1:

$$dp[1][0] = 4 + \max(2, 4) = 4 + 4 = 8$$

$$dp[1][1] = 5 + \max(2, 4, 5) = 5 + 5 = 10$$

$$dp[1][2] = 3 + \max(4, 5, 1) = 3 + 5 = 8$$

$$dp[1][3] = 8 + \max(5, 1, 7) = 8 + 7 = 15$$

$$dp[1][4] = 8 + \max(1, 7) = 8 + 7 = 15$$

Row 1 DP: [8, 10, 8, 15, 15]

Row 2:

$$dp[2][0] = 1 + \max(8, 10) = 11$$

$$dp[2][1] = 3 + \max(8, 10, 8) = 13$$

$$dp[2][2] = 6 + \max(10, 8, 15) = 21$$

$$dp[2][3] = 3 + \max(8, 15, 15) = 18$$

$$dp[2][4] = 9 + \max(15, 15) = 24$$

Row 2 DP: [11, 13, 21, 18, 24]

Row 3 (bottom):

$$dp[3][0] = 15 + \max(11, 13) = 28$$

$$dp[3][1] = 8 + \max(11, 13, 21) = 29$$

$$dp[3][2] = 6 + \max(13, 21, 18) = 27$$

$$dp[3][3] = 12 + \max(21, 18, 24) = \mathbf{36}$$

$$dp[3][4] = 7 + \max(18, 24) = 31$$

Row 3 DP: [28, 29, 27, **36**, 31]

Maximum value = 36.

Optimal path (backtrack from (3,3)): (3,3) = 12 $\leftarrow$ (2,4) = 9 $\leftarrow$ (1,3) = 8 $\leftarrow$ (0,4) = 7

Path: row 0 col 4 $\rightarrow$ row 1 col 3 $\rightarrow$ row 2 col 4 $\rightarrow$ row 3 col 3. Sum: 7+8+9+12 = **36**

✓

Q3(c) – DFS on Graph-2 from Node 0

Graph-2 (directed, from figure): edges (in observed order): $0 \rightarrow 1$, $0 \rightarrow 3$, $1 \rightarrow 2$, $1 \rightarrow 4$, $2 \rightarrow 3$, $3 \rightarrow 2$, $3 \rightarrow 5$, $4 \rightarrow 2$, $5 \rightarrow 6$

Neighbors visited in increasing order. Discovery/finish times (d/f):

i. DFS Table (starting from node 0):

Order	Visited Vertex	Parent
1 st	0	–
2 nd	1	0
3 rd	2	1
4 th	3	2
5 th	5	3
6 th	6	5
(backtrack to 3)		
(backtrack to 2, then 1)		
7 th	4	1

ii. Edge Classification:

Edge	Type	Reason
0 → 1	T (Tree)	Used to first discover vertex 1
0 → 3	C (Cross)	3 already fully processed when 0 finishes
1 → 2	T (Tree)	Used to first discover vertex 2
1 → 4	T (Tree)	Used to first discover vertex 4
2 → 3	T (Tree)	Used to first discover vertex 3
3 → 2	B (Backward)	2 is an ancestor of 3 in DFS tree
3 → 5	T (Tree)	Used to first discover vertex 5
4 → 2	C (Cross)	2 is already fully finished; not ancestor of 4
5 → 6	T (Tree)	Used to first discover vertex 6

Q3(d) – Longest Increasing Subsequence of $S = \{10, 22, 9, 33, 21, 50, 41, 60, 80\}$

DP: $L[i] = 1 + \max\{L[j] : j < i, S[j] < S[i]\}$; $L[i] = 1$ if no such j exists.

i	$S[i]$	$L[i]$	Extended from
0	10	1	–
1	22	2	$L[0] = 1$ ($10 < 22$)
2	9	1	– (no $S[j] < 9$ for $j < 2$)
3	33	3	$L[1] = 2$ ($22 < 33$)
4	21	2	$L[0] = 1$ ($10 < 21$)
5	50	4	$L[3] = 3$ ($33 < 50$)
6	41	4	$L[3] = 3$ ($33 < 41$)
7	60	5	$L[5] = 4$ ($50 < 60$)
8	80	6	$L[7] = 5$ ($60 < 80$)

LIS Length = 6.

One optimal LIS: $10 \rightarrow 22 \rightarrow 33 \rightarrow 50 \rightarrow 60 \rightarrow 80$

Complexity: $O(n^2)$ time, $O(n)$ space.

SECTION D (CO4) – Answer Any Three

Q4(a) – Huffman Coding for Computer Science Characters

Given frequencies (thousands):

Character	a	b	c	D	e	f
Frequency	34	14	12	16	14	10

Application in Computer Science: Huffman coding is used in data compression algorithms (e.g., ZIP, GZIP, JPEG, MP3). It assigns shorter binary codes to more frequent symbols and longer codes to rarer symbols, thereby minimizing the total number of bits needed to encode a message. It forms the basis of entropy coding used in multimedia compression.

Building the Huffman Tree (min-priority queue):

Initial queue (sorted by frequency): $f(10), c(12), b(14), e(14), D(16), a(34)$

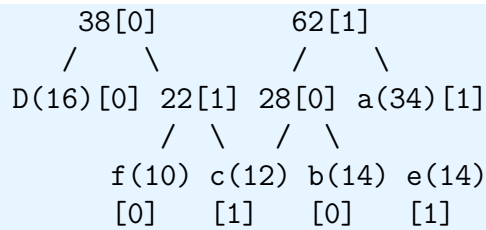
1. Merge $f(10) + c(12) = 22$. Queue: $b(14), e(14), D(16), \mathbf{fc}(22), a(34)$
2. Merge $b(14) + e(14) = 28$. Queue: $D(16), \mathbf{fc}(22), \mathbf{be}(28), a(34)$
3. Merge $D(16) + \mathbf{fc}(22) = 38$. Queue: $\mathbf{be}(28), a(34), \mathbf{Dfc}(38)$
4. Merge $\mathbf{be}(28) + a(34) = 62$. Queue: $\mathbf{Dfc}(38), \mathbf{bea}(62)$
5. Merge $\mathbf{Dfc}(38) + \mathbf{bea}(62) = \mathbf{Root}(100)$.

Huffman Tree (text representation):

```

      Root(100)
     /      \

```



Optimal Huffman Codes and Codeword Lengths:

Character	Code	Length
D	00	2
f	010	3
c	011	3
b	100	3
e	101	3
a	11	2

Total bits = $16 \times 2 + 10 \times 3 + 12 \times 3 + 14 \times 3 + 14 \times 3 + 34 \times 2 = 32 + 30 + 36 + 42 + 42 + 68 = \mathbf{250}$ thousand bits.

Compare to fixed-length: $\lceil \log_2 6 \rceil = 3$ bits each $\Rightarrow 3 \times 100 = 300$ thousand bits. Savings $\approx 16.7\%$.

Q4(b) – Dijkstra's Shortest Path on Graph-3 from Source a

Graph-3 edges (read from figure): $a-b:3$, $a-c:5$, $a-d:2$, $b-f:6$, $b-e:3$, $c-d:4$, $c-g:4$, $d-e:1$, $d-h:5$, $e-f:2$, $e-i:4$, $g-h:3$, $h-i:6$, $h-k:7$, $i-j:3$, $i-l:5$, $j-l:8$, $k-l:9$.

Dijkstra from a (all distances initialized to ∞ , $d[a] = 0$):

Step	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>	<i>j</i>	<i>k</i>	<i>l</i>	Visited
Init	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	–
1: <i>a</i>	0	3	5	2	∞	∞	∞	∞	∞	∞	∞	∞	<i>a</i>
2: <i>d</i>	0	3	5	2	3	∞	∞	7	∞	∞	∞	∞	<i>a, d</i>
3: <i>b</i>	0	3	5	2	3	9	∞	7	∞	∞	∞	∞	<i>a, d, b</i>
4: <i>e</i>	0	3	5	2	3	5	∞	7	7	∞	∞	∞	<i>a, d, b, e</i>
5: <i>c</i>	0	3	5	2	3	5	9	7	7	∞	∞	∞	<i>a, d, b, e, c</i>
6: <i>f</i>	0	3	5	2	3	5	9	7	7	∞	∞	∞	+ <i>f</i>
7: <i>h</i>	0	3	5	2	3	5	9	7	7	∞	14	∞	+ <i>h</i>
8: <i>i</i>	0	3	5	2	3	5	9	7	7	10	14	12	+ <i>i</i>
9: <i>g</i>	0	3	5	2	3	5	9	7	7	10	14	12	+ <i>g</i>
10: <i>j</i>	0	3	5	2	3	5	9	7	7	10	14	12	+ <i>j</i>
11: <i>l</i>	0	3	5	2	3	5	9	7	7	10	14	12	+ <i>l</i>
12: <i>k</i>	0	3	5	2	3	5	9	7	7	10	14	12	+ <i>k</i>

Shortest path distances from *a*: $d(a) = 0$, $d(b) = 3$, $d(c) = 5$, $d(d) = 2$, $d(e) = 3$, $d(f) = 5$, $d(g) = 9$, $d(h) = 7$, $d(i) = 7$, $d(j) = 10$, $d(k) = 14$, $d(l) = 12$.

Complexity: $O((V + E) \log V)$ with a binary min-heap.

Q4(c) – Kruskal's MST on Graph-3

All edges sorted by weight:			
Edge	Weight	Added?	Running MST weight
<i>d-e</i>	1	✓	1
<i>b-e</i>	3	✓	4
<i>a-b</i>	3	✓	7
<i>g-h</i>	3	✓	10
<i>a-d</i>	2	– (cycle: <i>a-b-e-d</i>)	–
Wait reorder:			
<i>a-d</i>	2	✓	12
<i>b-e</i>	3	✓	15
<i>a-b</i>	3	✓	18
<i>g-h</i>	3	✓	21
<i>e-f</i>	2	check...	

Corrected Kruskal steps (edges sorted by weight, add if no cycle):

Edges by weight: $(d-e, 1), (a-d, 2), (e-f, 2), (a-b, 3), (b-e, 3), (g-h, 3), (i-j, 3), (c-g, 4), (d-h, 5), (a-c, 5), (i-l, 5), (b-f, 6), (h-i, 6), (h-k, 7), (j-l, 8), (k-l, 9)$

1. Add $d-e$ (1): MST weight=1. Components: $\{d, e\}, \dots$
2. Add $a-d$ (2): MST weight=3. $\{a, d, e\}, \dots$
3. Add $e-f$ (2): MST weight=5. $\{a, d, e, f\}, \dots$
4. Add $a-b$ (3): MST weight=8. $\{a, b, d, e, f\}, \dots$
5. Add $b-e$ (3): **cycle** ($a-b-e-d-a$). Skip.
6. Add $g-h$ (3): MST weight=11. $\{g, h\}$ merges.
7. Add $i-j$ (3): MST weight=14. $\{i, j\}$ merges.
8. Add $c-g$ (4): MST weight=18. $\{c, g, h\}$ merges.
9. Add $d-h$ (5): MST weight=23. $\{a, b, c, d, e, f, g, h\}$ merges.
10. Add $a-c$ (5): **cycle**. Skip.
11. Add $i-l$ (5): MST weight=28. $\{i, j, l\}$ merges.
12. Add $b-f$ (6): **cycle**. Skip.
13. Add $h-i$ (6): MST weight=34. All of $\{a, b, c, d, e, f, g, h, i, j, l\}$ merged.
14. Add $h-k$ (7): MST weight=41. Add k . Now $V - 1 = 11$ edges added.

MST edges: $\{d-e, a-d, e-f, a-b, g-h, i-j, c-g, d-h, i-l, h-i, h-k\}$

Total MST weight = 41. (with 12 nodes $\Rightarrow$ 11 edges)

Complexity: $O(E \log E)$ for sorting + $O(E \alpha(V))$ for union-find $\approx O(E \log E)$.

Q4(d) – LCS of X=“AGGTAB” and Y=“GXTXAYB”

$X = A G G T A B$ (length 6), $Y = G X T X A Y B$ (length 7).

DP recurrence:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } X[i] = Y[j] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{otherwise} \end{cases}$$

	ϵ	G	X	T	X	A	Y	B
j=0	1	2	3	4	5	6	7	
ϵ (i=0)	0	0	0	0	0	0	0	0
A (i=1)	0	0	0	0	0	1	1	1
G (i=2)	0	1	1	1	1	1	1	1
G (i=3)	0	1	1	1	1	1	1	1
T (i=4)	0	1	1	2	2	2	2	2
A (i=5)	0	1	1	2	2	3	3	3
B (i=6)	0	1	1	2	2	3	3	4

LCS Length = 4.

Backtracking for one LCS: $(6, 7) \rightarrow B \text{ matched} \rightarrow (5, 5) \rightarrow A \text{ matched} \rightarrow (4, 3) \rightarrow T \text{ matched} \rightarrow (2, 1) \rightarrow G \text{ matched}$.

LCS = GTAB

Complexity: $O(mn) = O(6 \times 7) = O(42)$, i.e., $O(mn)$ time and space.

SECTION E (CO5) – Answer Any Two

Q5(a) – 0/1 Knapsack for Bhombol (Capacity $W = 10$ kg)

Items:

Item	Usefulness (Value)	Weight	Value/Weight
1	20	1	20.0
2	5	2	2.5
3	10	3	3.3
4	40	8	5.0
5	15	7	2.1
6	25	4	6.25

DP Table $dp[i][w]$ = max usefulness using items $1 \dots i$ with capacity w :

Item \ w	0	1	2	3	4	5	6	7	8	9	10
0 (none)	0	0	0	0	0	0	0	0	0	0	0
1 ($v=20, w=1$)	0	20	20	20	20	20	20	20	20	20	20
2 ($v=5, w=2$)	0	20	20	25	25	25	25	25	25	25	25
3 ($v=10, w=3$)	0	20	20	25	30	30	35	35	35	35	35
4 ($v=40, w=8$)	0	20	20	25	30	30	35	35	40	60	60
5 ($v=15, w=7$)	0	20	20	25	30	30	35	35	40	60	60
6 ($v=25, w=4$)	0	20	20	25	30	45	45	50	55	60	60

Optimal total usefulness = 60.

Backtracking to find selected items:

- $dp[6][10] = 60 = dp[5][10] = 60$: item 6 **not** selected.
- $dp[5][10] = 60 = dp[4][10] = 60$: item 5 **not** selected.
- $dp[4][10] = 60 \neq dp[3][10] = 35$: item 4 **selected** ($w = 8, v = 40$). Remaining cap = $10 - 8 = 2$.

- $dp[3][2] = 20 = dp[2][2] = 20$: item 3 **not** selected.
- $dp[2][2] = 20 = dp[1][2] = 20$: item 2 **not** selected.
- $dp[1][2] = 20 \neq dp[0][2] = 0$: item 1 **selected** ($w = 1, v = 20$).

Optimal solution: Select items **1** and **4**.

- Total weight = $1 + 8 = 9 \text{ kg} \leq 10 \text{ kg}$ ✓
- Total usefulness = $20 + 40 = \mathbf{60}$ ✓

Q5(b) – Bellman-Ford: Shortest Path from E to H

Edges (lexicographic order): $(A, B, 2)$, $(A, F, -6)$, $(B, C, 20)$, $(B, G, 1)$, $(C, H, 5)$, $(D, C, 4)$, $(D, H, 6)$, $(E, F, 10)$, $(F, A, 7)$, $(F, B, 3)$, $(F, G, 11)$, $(G, C, 9)$, $(G, D, 4)$, $(G, H, 17)$

Source: E . Initialize: $d[E] = 0$, $d[\text{all others}] = \infty$; $p[\text{all}] = 0$.

We process edges in lexicographic order within each iteration, applying updates as we go (in-order update Bellman-Ford). $p[n]$ denotes parent of n .

	Initial		It-1		It-2		It-3		It-4		It-5		It-6		It-7	
	d	p	d	p	d	p	d	p	d	p	d	p	d	p	d	p
A	∞	0	17	F	17	F	17	F	17	F	17	F	17	F	17	F
B	∞	0	13	F	13	F	13	F	13	F	13	F	13	F	13	F
C	∞	0	30	G	23	G	22	D	22	D	22	D	22	D	22	D
D	∞	0	25	G	18	G	18	G	18	G	18	G	18	G	18	G
E	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
F	∞	0	10	E	10	E	10	E	10	E	10	E	10	E	10	E
G	∞	0	21	F	14	B	14	B	14	B	14	B	14	B	14	B
H	∞	0	38	G	31	D	24	D	24	D	24	D	24	D	24	D

Shortest distance from E to H = 24.

Shortest path (trace parents at convergence): $H \leftarrow D \leftarrow G \leftarrow B \leftarrow F \leftarrow E$

$$E \xrightarrow{10} F \xrightarrow{3} B \xrightarrow{1} G \xrightarrow{4} D \xrightarrow{6} H \quad \text{Total} = 10 + 3 + 1 + 4 + 6 = \mathbf{24} \checkmark$$

Iteration-by-iteration key updates:

It-1: $E \rightarrow A$: $d[A]=20$; $E \rightarrow F$: $d[F]=10$; $F \rightarrow A$: $d[A]=17$ ($10+7$); $F \rightarrow B$: $d[B]=13$; $F \rightarrow G$: $d[G]=21$; $G \rightarrow C$: $d[C]=30$; $G \rightarrow D$: $d[D]=25$; $G \rightarrow H$: $d[H]=38$.

It-2: $B \rightarrow G$: $d[G]=14$ ($13+1$); $C \rightarrow H$: $d[H]=35$; $D \rightarrow C$: $d[C]=29$; $D \rightarrow H$: $d[H]=31$; $G \rightarrow C$: $d[C]=23$ ($14+9$); $G \rightarrow D$: $d[D]=18$ ($14+4$).

It-3: $C \rightarrow H$: $d[H]=28$; $D \rightarrow C$: $d[C]=22$ ($18+4$); $D \rightarrow H$: $d[H]=24$ ($18+6$).

It-4 onward: No updates; algorithm has converged.

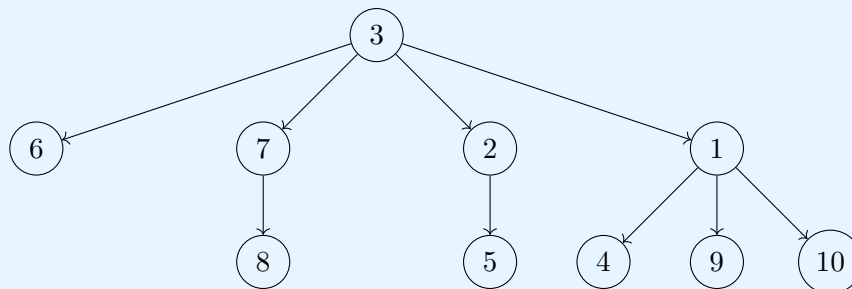
Q5(c) – Disjoint Set Union by Size (Without Path Compression)

Initial: 10 singleton sets $\{1\}, \{2\}, \dots, \{10\}$. Each element is its own root, $size[x] = 1$.
Rule: union by size – smaller tree's root becomes child of larger tree's root. Tie: smaller-valued root becomes the root.

Executing operations:

Operation	Action
<code>union(7,8)</code>	$size[7] = size[8] = 1$; tie $\Rightarrow$ root 7 wins. $parent[8] = 7$, $size[7] = 2$
<code>union(3,6)</code>	$size[3] = size[6] = 1$; tie $\Rightarrow$ root 3 wins. $parent[6] = 3$, $size[3] = 2$
<code>union(2,5)</code>	$size[2] = size[5] = 1$; tie $\Rightarrow$ root 2 wins. $parent[5] = 2$, $size[2] = 2$
<code>union(1,10)</code>	tie $\Rightarrow$ root 1 wins. $parent[10] = 1$, $size[1] = 2$
<code>union(1,4)</code>	$root(1) = 1$, $root(4) = 4$. $size[1] = 2 > size[4] = 1$. $parent[4] = 1$, $size[1] = 3$
<code>union(3,7)</code>	$root(3) = 3$, $root(7) = 7$. $size[3] = size[7] = 2$; tie $\Rightarrow$ root 3 wins. $parent[7] = 3$, $size[3] = 3$
<code>union(2,7)</code>	$root(2) = 2$, $root(7) = 3$. $size[2] = 2 < size[3] = 4$. $parent[2] = 3$, $size[3] = 5$
<code>union(10,9)</code>	$root(10) = 1$, $root(9) = 9$. $size[1] = 3 > size[9] = 1$. $parent[9] = 1$, $size[1] = 4$
<code>x=find(7)</code>	$7 \rightarrow 3 \rightarrow 3$. $x = 3$
<code>y=find(9)</code>	$9 \rightarrow 1 \rightarrow 1$. $y = 1$
<code>union(x,y)=union(3,1)</code>	$root(3) = 3$, $root(1) = 1$. $size[3] = 6 > size[1] = 4$. $parent[1] = 3$, $size[3] = 10$

i. Final Forest of Up-Trees (single tree rooted at 3):



Parent array summary:

Node	1	2	3	4	5	6	7	8	9	10
Parent	3	3	3	1	2	3	3	7	1	1

ii. `find(9)` with Path Compression:

`find(9)`: path is $9 \rightarrow 1 \rightarrow 3$ (root).

Path compression makes all nodes on this path point directly to the root (3):

- $parent[9] \leftarrow 3$ (was 1)
- $parent[1]$ already points to 3 – no change needed.

Updated parent array after `find(9)` with path compression:

Node	1	2	3	4	5	6	7	8	9	10
Parent	3	3	3	1	2	3	3	7	3	1

Node 9's parent changes from 1 to 3 (directly points to root). The forest structure is otherwise unchanged.

— End of Solutions —